Rider App Client Flow Guide

Prepared for client review Project: Enatega Deliveries Rider App Document date: June 19, 2026

1. Purpose of This Document

This guide explains how the Rider App works from a business and module-flow perspective. It is intended to

- the rider journey from login to completed delivery
- how the major app modules work together
- which backend services the app depends on
- which operational details are important for launch, testing, and support

2. Solution Overview

The Rider App is the delivery partner application used by riders to log in, receive available orders, accept and process deliveries, communicate during active orders, review earnings, manage wallet withdrawals, and maintain operational profile settings.

At a high level, the app works in this sequence:

- the app starts and restores any saved rider session
- unauthenticated riders see the splash and login flow
- authenticated riders land in the main rider experience with four core tabs:
- Home
- Wallet
- Earnings
- Profile
- the Home module is connected to real-time order events through sockets
- order actions update backend status and refresh the rider dashboard
- profile and settings modules let the rider maintain operational data needed for delivery work

3. End-to-End Rider Journey

3.1 App Launch

When the app opens, it loads its shared providers in this order:

- theme provider
- safe area provider
- TanStack Query provider
- localization provider
- authentication provider

The authentication provider restores the saved token, refresh token, and rider identity from secure device st

- if a valid token exists, the rider is taken into the main app
- if no token exists, the rider remains in the authentication flow

3.2 Splash and Login

The authentication flow contains:

- Splash screen
- Login screen

The splash screen is a short branded startup animation. It then routes the rider to login.

On login:

- the rider enters email and password
- the app validates the input locally
- the app attempts to collect an Expo push token from the device
- the login request is sent to the rider authentication endpoint
- on success, the access token, refresh token, and rider profile are saved securely
- the main rider app opens automatically

3.3 Main Rider Workspace

Once authenticated, the rider enters the main workspace. The primary navigation consists of four bottom tab

- Home
- Wallet
- Earnings
- Profile

Additional detail and settings screens are opened through stack navigation on top of these tabs, such as:

- order detail
- order chat
- earnings detail
- deliveries detail
- profile details
- language
- vehicle type
- bank management
- work schedule

4. Module-by-Module Functional Flow

4.1 Home Module

The Home module is the rider’s operational dashboard. It displays delivery work grouped into three tabs:

- New
- Processing
- Delivered

The Home module uses a summary API plus paginated order lists. The summary provides counters for:

- new orders
- processing orders
- delivered orders

The order list is fetched separately for each tab, which keeps the experience responsive and allows large lists to be loaded page by page.

Important behavior:

- if the rider profile is marked as not approved, the app blocks order access
- in that state, the rider is shown a pending approval message and can log out

4.2 Real-Time Order Sync

The Rider App includes a real-time socket connection for order activity. Once a rider is logged in:

- the app opens a socket connection using the rider token
- the rider user ID is registered on the socket
- the app listens for events such as:
- order-status-updated
- rider-status-updated
- rider-order-available
- affected query caches are invalidated or updated immediately
- a new-order beep is started or stopped depending on current new-order counts

The socket layer also reconnects when:

- the app returns to the foreground
- internet connectivity is restored

This ensures the Home dashboard stays in sync with dispatch activity as conditions change.

4.3 Order Assignment and Processing

When a rider selects an active order, the app opens the order detail and processing flow.

This module is responsible for:

- showing order detail
- showing delivery progress state
- allowing rider status transitions
- opening navigation
- calling the customer
- opening order chat

The delivery flow follows a controlled status progression. The screen derives the current progress stage and

- heading to store
- arrived at store
- waiting for order
- picked up
- out for delivery
- arrived at customer
- delivered

When the rider updates the delivery status:

- the update is sent to the backend
- the order detail query is refreshed
- Home summary and order lists are invalidated
- socket-driven updates keep the rest of the app synchronized

4.4 Order Chat Module

The order chat screen supports rider communication during an active order.

Its flow is hybrid:

- historical messages are loaded through the API
- live message delivery is handled through a socket connection

- outgoing messages are sent through both socket emission and API persistence

This design gives the app two benefits:

- the rider sees live updates quickly
- the conversation remains recoverable after reopening the screen

The chat module also updates order detail state when a new chat box is created during the first message exchange.

4.5 Wallet Module

The Wallet tab gives riders visibility into their delivery funds and payout activity.

The Wallet module includes:

- current balance display
- available withdrawal amount
- paginated transaction history
- withdrawal request submission

The rider can initiate a withdrawal from the wallet screen. On successful submission:

- the wallet queries are invalidated
- balance and history refresh automatically
- a success confirmation is shown to the rider

4.6 Earnings Module

The Earnings tab shows performance-oriented income information.

It includes:

- earnings chart data
- recent earning activity
- navigation to deeper earnings detail

This module is intended to help riders monitor short-term and historical earning patterns based on backend-provided summary data.

4.7 Profile Module

The Profile tab acts as the rider’s account and operations center.

It provides access to:

- profile overview
- profile details
- password update
- document visibility and updates
- language settings
- vehicle type settings
- bank management
- work schedule
- external informational links
- logout

The profile details flow also supports rider document maintenance, including:

- driving license number and images
- vehicle plate number and images

Document uploads are sent as multipart form-data to the backend.

4.8 Language Module

The language settings screen:

- loads available language options from the backend
- shows the selected language
- lets the rider update the backend language preference
- updates the in-app language when the chosen language is supported locally

At present, local app translations are configured for:

- English
- French

4.9 Vehicle Type Module

The vehicle type module:

- loads available delivery vehicle options from the backend
- highlights the rider’s selected type
- lets the rider update their active delivery vehicle

This is important where dispatch logic, capacity rules, or rider presentation depend on vehicle type.

4.10 Bank Management Module

The bank management module allows the rider to maintain payout-related account details, including:

- bank name
- account title
- account number
- IBAN
- currency
- account code

Client-side validation ensures key required fields are present before submission.

4.11 Work Schedule Module

The work schedule module lets the rider manage weekly availability by day.

Each day can be:

- activated or deactivated
- configured with one or more open and close time slots

This supports operational scheduling scenarios where riders work split shifts or non-uniform weekly schedules.

5. System Flow Behind the Screens

The app follows a clear internal flow:

- screen layer presents rider-facing UI
- hooks layer manages queries and mutations
- service layer handles API communication
- auth layer manages secure session persistence
- socket layer handles real-time order and chat updates

This separation makes the app easier to maintain and helps ensure:

- reusable business logic
- centralized API handling
- consistent caching
- secure token management

6. Backend Dependencies

For full production behavior, the Rider App depends on working backend support for:

- rider authentication
- session-based authenticated API access
- rider home summary
- new, processing, and delivered order lists
- order detail retrieval
- order assignment
- rider order status updates
- wallet balance
- wallet history
- withdrawal requests
- earnings summary and activity
- rider profile data
- password update
- rider document upload
- language settings
- vehicle type settings
- bank details
- work schedule
- support chat history
- support chat send message
- socket-based order events
- socket-based chat events

Environment variables currently expected by the app include:

- ‘EXPO_PUBLIC_API_BASE_URL‘ (required)
- ‘EXPO_PUBLIC_SOCKET_URL‘ (optional override)
- ‘EXPO_PUBLIC_SOCKET_PATH‘ (optional override)

7. Important Operational Notes for the Client

7.1 Session Handling

The Rider App is token-driven. A rider is considered logged in when a valid access token is available. Tokens and rider identity are stored securely on the device.

If the backend returns a session-expiry style response, the app can automatically clear the session and return the rider to the authentication flow.

7.2 Push Token Collection

During login, the app attempts to register an Expo push token so the rider device can be targeted for push notifications where supported.

Important note:

- Expo push token retrieval requires a physical device
- simulators and emulators do not provide the same production behavior

7.3 Real-Time Experience Depends on Socket Availability

The live order experience is strongest when the socket server is available and reachable. Without socket connectivity, riders can still use API-driven screens, but real-time updates will be delayed until the next refresh cycle.

7.4 Rider Approval Status Matters

The app checks whether the rider profile is approved. If approval is false, order access is blocked on the Home screen. This is an important business-control mechanism for onboarding and compliance.

7.5 Current Map and Navigation Limitation

The current order detail screen uses fallback pickup and delivery coordinates in the map and navigation action. This means the visual route and map launch behavior are not yet tied to live order coordinates from the backend.

This is an important rollout note for the client:

- order-processing status updates are implemented
- map/navigation behavior is present
- live location wiring for actual pickup and delivery coordinates still needs to be completed if production routing accuracy is required

7.6 Current Availability Toggle Scope

The availability switch visible in profile-related UI is currently handled locally in the app interface. It is not yet wired to a dedicated backend rider-availability update flow in the current codebase.

If the client expects dispatcher-facing online or offline rider availability control, that integration should be confirmed separately.

7.7 Localization Scope

Backend language settings are supported, but the app-side translation bundle is currently configured for English and French only. Additional languages will require app localization resources to be added.

8. Recommended Client Testing Scenarios

The following scenarios are especially important during UAT or pilot rollout:

- rider login with valid credentials
- rider login with invalid credentials
- session restore after app restart
- rider blocked due to non-approved profile
- new order appearing in Home via socket
- rider assigning or accepting an order
- rider moving through each delivery status
- rider opening chat and sending messages
- wallet balance display
- withdrawal request submission

- earnings chart and recent activity display
- profile document upload
- bank details update
- language update
- vehicle type update
- work schedule update with multiple time slots
- forced logout on expired session

9. Summary

The Rider App is structured as a delivery operations application for riders, centered around:

- secure login and session management
- real-time order visibility
- controlled delivery-status progression
- communication during active orders
- wallet and earnings visibility
- operational profile and settings maintenance

From a client perspective, the core order lifecycle, profile settings, wallet flow, and real-time foundations are all present. The main implementation notes to keep in mind are the current fallback map coordinates, the local-only availability toggle behavior, and the current two-language in-app localization scope.